

OSY Summer 23 Question Paper

1. Attempt any FIVE of the following: 10

a) State any two features of Linux.

Ans.

- Open source: The source code for Linux is free and available.
- Multitasking: Linux can run multiple applications at once.
- Customization: Linux is highly customizable, allowing users to tailor the operating system to their needs.
- Portable: Linux software can work on different hardware in the same way.
- Inexpensive: Linux is free to download, and upgrades are also free.
- File system: Linux file systems operate from a single namespace, with all files existing under the root directory, “/”.
- Wide range of distributions: There are many Linux distributions, each designed for specific use cases and preferences.

b) Difference between Time sharing system and Real time system (any 2 points)

Ans.

S	Time Sharing Operating System	Real-Time Operating System
1	In time sharing operating system , quick response is emphasized for a request.	While in real time operating system , computation tasks are emphasized before its nominative point.
2	In this operating system Switching method/function is available.	While in this operating system Switching method/function is not available.
3	In this operating system any modification in the program can be possible.	While in this modification does not take place.
4	In this OS, computer resources are shared to the external.	But in this OS, computer resources are not shared to the external.
5	It deals with more than processes or applications simultaneously.	Whereas it deals with only one process or application at a time.
6	In this OS, the response is provided to the user within a second.	While in real time OS, the response is provided to the user within time constraint.
7	In time sharing system, high priority tasks can be preempted by lower priority tasks, making it impossible to guarantee a response time for your critical applications.	Real time operating systems, give users the ability to prioritize tasks so that the most critical task can always take control of the process when needed.

c) State any four services of operating system.

Ans.

- Process management: Manages the creation and termination of processes

- Memory management: Ensures that programs have enough memory to run
- File management: Allows users to create, delete, modify, and organize files and directories
- Security: Protects the system from unauthorized access and threats
- Multitasking: Allows multiple applications to run simultaneously
- Network management: Manages network connections and communications

d) Write the difference between pre-emptive and non-preemptive scheduling.

Ans.

Parameter	PREEMPTIVE SCHEDULING	NON-PREEMPTIVE SCHEDULING
Basic	In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state
Interrupt	Process can be interrupted in between.	Process can not be interrupted until it terminates itself or its time is up
Starvation	If a process having high priority frequently arrives in the ready queue, a low priority process may starve	If a process with a long burst time is running CPU, then later coming process with less CPU burst time may starve
Overhead	It has overheads of scheduling the processes	It does not have overheads
Flexibility	flexible	Rigid
Cost	Cost associated	No cost associated
CPU Utilization	In preemptive scheduling, CPU utilization is high	It is low in non preemptive scheduling
Waiting Time	Preemptive scheduling waiting time is less	Non-preemptive scheduling waiting time is high
Response Time	Preemptive scheduling response time is less	Non-preemptive scheduling response time is high
Decision making	Decisions are made by the scheduler and are based on priority and time slice allocation	Decisions are made by the process itself and the OS just follows the process's instructions
Process control	The OS has greater control over the scheduling of processes	The OS has less control over the scheduling of processes
Over	Higher overhead due to frequent	Lower overhead since context switching is less

head	context switching	frequent
Exa mple s	Examples of preemptive scheduling are Round Robin and Shortest Remaining Time First	Examples of non-preemptive scheduling are First Come First Serve and Shortest Job First

e) Define following terms :

i) Virtual Memory

ii) Paging

Ans.Virtual Memory:Virtual memory is a memory management capability of an operating system (OS) that uses hardware and software to allow a computer to compensate for physical memory shortages by temporarily transferring data from random access memory (RAM) to disk storage.

Paging:Paging is a storage mechanism used in OS to retrieve processes from secondary storage to the main memory as pages. The primary concept behind paging is to break each process into individual pages. Thus the primary memory would also be separated into frames.

f) List any four file attributes and it's meaning.

Ans.

1. **Name:** File name is the name given to the file. A name is usually a string of characters.
2. **Identifier:** Identifier is a unique number for a file. It identifies files within the file system. It is not readable to us, unlike file names.
3. **Type:** Type is another attribute of a file which specifies the type of file such as archive file (.zip), source code file (.c, .java), .docx file, .txt file, etc.
4. **Location:** Specifies the location of the file on the device (The directory path). This attribute is a pointer to a device.
5. **Size:** Specifies the current size of the file (in Kb, Mb, Gb, etc.) and possibly the maximum allowed size of the file.
6. **Protection:** Specifies information about Access control (Permissions about Who can read, edit, write, and execute the file.) It provides security to sensitive and private information.
7. **Time, date, and user identification:** This information tells us about the date and time on which the file was created, last modified, created and modified by which user, etc.

g) Define Deadlock.

Ans.Deadlock is a situation in computing where two or more processes are unable to proceed because each is waiting for the other to release resources. Key concepts include mutual exclusion, resource holding, circular wait, and no preemption.

Consider an example when two trains are coming toward each other on the same track and there is only one track, none of the trains can move once they are in front of each other. This is a practical example of deadlock.

2.Attempt any THREE of the following: 12

a) Explain different types of system calls.

Ans. System call provides an interface between a running program and operating system. It allows user to access services provided by operating system. These system calls are procedures written using C, C++ and assembly language instructions. Each operating system has its own name for each system call. Each system call is associated with a number that identifies itself.

System calls:

Process Control: Program in execution is a process. A process to be executed must be loaded in main memory. While executing it may need to wait, terminate or create & terminate child processes.

- i, • end, abort
- i, • load, execute
- i, • create process, terminate process
- get process attributes, set process attributes
- i, • wait for time
- i, • wait event, signal event
- i, • allocate and free memory

File Management: System allows us to create and delete files. For create and delete operation system call requires the name of the file and other attributes of the file. File attributes include file type, file size, protection codes, accounting information and so on. Systems access these attributes for performing operations on file and directories. Once the file is created, we can open it and use it. System also allows performing reading, writing or repositioning operations on file.

- i, • create file, delete file
- i, • open, close
- i, • read, write, reposition
- i, • get file attributes, set device attributes
- i, • logically attach or detach devices

3. Device Management: When a process is in running state, it requires several resources to execute. These resources include main memory, disk drives, files and so on. If the resource is available, it is assigned to the process. Once the resource is allocated to the process, process can read, write and reposition the device.

- i, • request device, release device
- i, • read, write, reposition
- i, • get device attributes, set device attributes
- i, • logically attach or detach devices

4. Information Maintenance: Transferring information between the user program and the operating system requires system call. System information includes displaying current date and time, the number of current user, the version number of the operating system, the amount of free memory or disk space and so on. Operating system keeps information about all its processes that can be accessed with system calls such as get process attributes and set process attributes.

- i, • get time or date, set time or date
- i, • get system data, set system data
- i, • get process, file, or devices attributes
- i, • set process, file, or devices attributes

5. Communication: Processes in the system, communicate with each other. Communication is done by using two models: message passing and shared memory. For transferring messages, sender process connects itself to receiving process by specifying receiving process name or identity. Once the communication is over system close the connection between communicating processes.

- i, • create, delete communication connection
- i, • send, receive messages
- i, • transfer status information
- i, • attach or detach remote devices.

b) Draw and explain process control block in detail.

Ans. Each process is represented as a process control block (PCB) in the operating system. It contains information associated with specific process.

Process State: It indicates current state of a process. Process state can be new, ready, running, waiting and terminated.

Process number: Each process is associated with a unique number which is known process identification number.

Program Counter: It indicates the address of the next instruction to be executed for the process.

CPU Registers: The registers vary in number and type depending on the computer architecture. Register includes accumulators, index registers, stack pointers and general purpose registers plus any condition code information.

Memory Management Information: It includes information such as value of base and limit registers, page tables, segment tables, depending on the memory system used by OS.

Accounting Information: This information includes the amount of

CPU used, time limits, account holders, job or process number and so on. It also includes information about listed I/O devices allocated to the process such as list of open files.

Each PCB gives information about a particular process for which it is designed.

c) State and explain four scheduling criteria.

Ans.1. CPU utilization: In multiprogramming the main objective is to keep CPU as busy as possible. CPU utilization can range from 0 to 100 percent.

2.Throughput: It is the number of processes that are completed per unit time. It is a measure of work done in the system. When CPU is busy in executing processes, then work is being done in the system. Throughput depends on the execution time required for any process. For long processes, throughput can be one process per unit time whereas for short processes it may be 10 processes per unit time.

3. Turnaround time: The time interval from the time of submission of a process to the time of completion of that process is called as turnaround time. It is the sum of time period spent waiting to get into the memory, waiting in the ready queue, executing with the CPU, and doing I/O operations.

4.Waiting time: It is the sum of time periods spent in the ready queue by a process. When a process is selected from job pool, it is loaded into the main memory (ready queue). A process waits in ready queue till CPU is allocated to it. Once the CPU is allocated to the process, it starts its execution and if required request for resources. When the resources are not available that process goes into waiting state and when I/O request completes, it goes back to ready queue. In ready queue again it waits for CPU allocation.

5.Response time: The time period from the submission of a request until the first response is produced is called as response time. It is the time when system responds to the process request not the completion of a process. In the system, a process can Produce some output fairly early and can continue computing new results while previous results are being output to the user.

d) Define fragmentation. Explain Internal and External Fragmentation

Ans.An unwanted problem with operating systems is fragmentation, which occurs when processes load and unload from memory and divide available memory. Because memory blocks

are so small, they cannot be assigned to processes, and thus remain idle. It's also important to realize that programs create free space or holes in memory when they are loaded and unloaded. Because additional processes cannot be assigned to these little pieces, memory is used inefficiently.

The memory allocation scheme determines the fragmentation circumstances. These regions of memory become fragmented when the process loads and unloads from it, making it unusable for incoming processes. We refer to it as fragmentation.

1. Internal Fragmentation

Internal fragmentation occurs when there is unused space within a memory block. For example, if a system allocates a 64KB block of memory to store a file that is only 40KB in size, that block will contain 24KB of internal fragmentation. When the system employs a fixed-size block allocation method, such as a memory allocator with a fixed block size, this can occur.

2. External Fragmentation

External fragmentation occurs when a storage medium, such as a hard disc or solid-state drive, has many small blocks of free space scattered throughout it. This can happen when a system creates and deletes files frequently, leaving many small blocks of free space on the medium. When a system needs to store a new file, it may be unable to find a single contiguous block of free space large enough to store the file and must instead store the file in multiple smaller blocks. This can cause external fragmentation and performance problems when accessing the file.

Fragmentation can also occur at various levels within a system. File fragmentation, for example, can occur at the file system level, in which a file is divided into multiple non-contiguous blocks and stored on a storage medium. Memory fragmentation can occur at the [memory management](#) level, where the system allocates and deallocates memory blocks dynamically. Network fragmentation occurs when a packet of data is divided into smaller fragments for transmission over a network.

3. Attempt any THREE of the following: 12

a) Describe multiprocessor OS with its advantages. (any two)

Ans.

A **Multiprocessor Operating System** (also known as a **multiprocessing OS**) is designed to manage and coordinate the execution of multiple processors (CPUs) within a single computer system. In a multiprocessor system, multiple CPUs work together in a tightly coupled manner, sharing resources such as memory, input/output devices, and system buses. The main objective of a multiprocessor OS is to efficiently manage the processors, ensure that tasks are distributed among them, and optimize resource utilization to improve performance and reliability.

1. Overview of Multiprocessor Systems

a. Types of Multiprocessor Systems

Multiprocessor systems can be classified based on how they are structured and how the processors communicate with one another:

1. **Symmetric Multiprocessing (SMP):**

- a. In **SMP**, all processors are treated equally. Each processor has equal access to memory, I/O devices, and the system bus, and all processors run a single operating system instance.
- b. Every processor can execute any process or thread, and the OS dynamically balances the load among the processors.
- c. SMP systems are commonly used in servers and high-performance computers.

2. **Example:** A quad-core desktop computer running Windows or Linux is an example of an SMP system, where all four cores (processors) share the same memory and resources.

3. **Asymmetric Multiprocessing (AMP):**

- a. In **AMP**, processors are assigned specific tasks. One processor acts as the **master**, controlling the system, while the others act as **slaves** and handle specific tasks assigned to them.
- b. The master processor runs the OS and assigns jobs to the slave processors, which do not have direct access to system resources like the master.
- c. AMP systems are used in simpler or embedded systems where task specialization is beneficial.

4. **Example:** In embedded systems like a smart appliance, one processor might manage system control while others handle real-time tasks like sensor data collection.

5. **Distributed Systems:**

- a. In a **distributed system**, multiple processors are connected over a network, each running its own OS instance. However, in this case, the processors are not tightly coupled, and they operate independently, unlike in SMP or AMP.

b. Multiprocessing OS Features

A multiprocessor OS is designed with several key capabilities to support multiple processors:

- **Parallel Processing:** It enables multiple CPUs to execute processes or threads simultaneously, significantly improving throughput.
- **Concurrency Control:** The OS manages shared resources and synchronizes access to prevent conflicts and ensure data integrity.
- **Load Balancing:** The OS distributes tasks and processes across available processors to optimize performance.
- **Scalability:** It scales well with additional processors, meaning that performance increases as more processors are added.

2. Advantages of Multiprocessor OS

Multiprocessor operating systems offer a range of advantages, particularly in performance, scalability, and reliability. Here are some of the key benefits:

a. Increased Performance and Throughput

- **Parallelism:** In a multiprocessor system, tasks are executed in parallel, which significantly increases the speed of computation and system throughput. Multiple processes or threads can run at the same time on different processors.
- **Faster Execution of Multithreaded Applications:** Applications that are designed to run in parallel (multithreaded applications) benefit greatly from multiprocessor systems. Threads from the same application can be distributed across processors, leading to faster completion times.
- **Reduced Waiting Time:** Since multiple processors can handle different tasks simultaneously, the overall waiting time for executing processes is reduced. This is especially useful for high-performance computing tasks like scientific simulations, machine learning, and data analysis.

b. Improved Reliability and Fault Tolerance

- **Redundancy:** If one processor fails in a multiprocessor system, the system can continue to function by redistributing the workload to the remaining processors. This makes the system more reliable and fault-tolerant compared to single-processor systems.
- **Graceful Degradation:** In case of hardware failure or overload, the system doesn't crash entirely. Instead, it slows down as the workload is distributed across fewer processors, providing better fault tolerance.
- **Load Sharing:** Tasks can be distributed among available processors, ensuring that no single processor is overburdened, which enhances system stability.

c. Scalability

- **Scalable Performance:** One of the biggest advantages of multiprocessor systems is their ability to scale performance by adding more processors. If the system needs to handle more workload, additional CPUs can be added to improve performance.
- **Support for Large Applications:** Multiprocessor OSs can efficiently support large applications that require significant computational power. For instance, applications in fields like weather forecasting, physics simulations, or complex financial modeling benefit from the additional computing resources.

d. Efficient Resource Utilization

- **Resource Sharing:** In a multiprocessor system, CPUs share memory, I/O devices, and other system resources, which reduces idle time for hardware components. This leads to better resource utilization compared to single-processor systems, where resources may sit idle while waiting for the CPU.
- **Cost Efficiency:** A multiprocessor system can be more cost-effective in high-performance environments because a single system with multiple processors may be less expensive and more efficient than maintaining multiple independent systems.

e. Faster Response Time

- **Reduced Latency:** Multiprocessor systems provide faster response times for interactive applications, especially when there are multiple tasks or users. Each processor can handle a different request, resulting in quicker responses.

f. Support for Multiuser Environments

- **Concurrent User Support:** Multiprocessor OSs can better handle multiple users simultaneously. Each processor can work on a different user's process, ensuring that each user experiences less delay or slowdown, which is critical for servers and shared computing environments.

b) Explain different components of operating system.

Ans.1. Process management

2. Main memory management

3. File management

4. I/O system management

5. Secondary storage management

1.Process Management:

A program is a set of instructions. When CPU is allocated to a program, it can start its execution. A program in execution is a process. A word processing program run by a user on a PC is a process. A process needs various system resources including CPU time, memory, files and I/O devices to complete the job execution. These resources can be given to the process when it is created or allocated to it while it is running

The operating system responsible for the following **activities** in connection with process management:

- Creation and deletion of user and system processes.
- Suspension and resumption of processes.
- A mechanism for process synchronization.
- A mechanism for process communication.
- A mechanism for deadlock handling.

2. Main-Memory Management

Main memory is a large array of words or bytes, ranging in size from hundreds of thousands to billions. Each word or byte has its own address. Main memory is a repository of quickly accessible data shared by the CPU and I/O devices. The central processor reads instructions from main memory during the instruction fetch cycle and both reads and writes data from main memory during the data fetch cycle. The main memory is generally the only large storage device that the CPU is able to address and access directly.

The operating system responsible for the following **activities** in connection with main memory's management:

- Keeping track of which parts of memory are currently being used and by whom.

ï, · Deciding which processes (or parts thereof) and data to move into and out of memory. 3. Allocating and deallocating memory space as needed.

3. File Management

A file is a collected of related information defined by its creator. Computer can store files on the disk (secondary storage), which provide long term storage. Some examples of storage media are magnetic tape, magnetic disk and optical disk. Each of these media has its own properties like speed, capacity, and data transfer rate and access methods. A file system normally organized into directories to ease their use. These directories may contain files and other directions.

The operating system responsible for the following **activities** in connection with file management:

- ï, · The creation and deletion of files.
- ï, · The creation and deletion of directions.
- ï, · The support of primitives for manipulating files and directions.
- ï, · The mapping of files onto secondary storage.
- ï, · The backup of files on stable storage media.

4. I/O device Management

Input / Output device management provides an environment for the better interaction between system and the I / O devices (such as printers, scanners, tape drives etc.). To interact with I/O devices in an effective manner, the operating system uses some special programs known as device driver. The device drivers take the data that operating system has defined as a file and then translate them into streams of bits or a series of laser pulses (in regard with laser printer).

The I/O subsystem consists of several components:

- ï, · A memory management component that includes buffering, caching, spooling
- ï, · A general device driver interface
- ï, · Drivers for specific hardware devices

5. Secondary-Storage Management

The computer system provides secondary storage to back up main memory. Secondary storage is required because main memory is too small to accommodate all data and programs, and the data that it holds is lost when power is lost. Most of the programs including compilers, assemblers, word processors, editors, and formatters are stored on a disk until loaded into memory. Secondary storage consists of tapes drives, disk drives, and other media.

The operating system is responsible for the following **activities** in connection with disk management:

- i, · Free space management
- ii, · Storage allocation
- iii, · Disk scheduling.

c) Explain different types of schedulers.

Ans. There are three types of scheduler:

- i, · Long term scheduler
- ii, · Short term scheduler
- iii, · Medium term scheduler

1. Long term scheduler: It selects programs from job pool and loads them into the main memory. It controls the degree of multiprogramming. The degree of multiprogramming is the number of processes loaded (existing) into the main memory. System contains I/O bound processes and CPU bound processes. An I/O bound process spends more time for doing I/O operations whereas CPU bound process spends more time in doing computations with the CPU. So It is the responsibility of long term scheduler to balance the system by loading some I/O bound and some CPU bound processes into the main memory. Long term scheduler executes only when a process leaves the system, so it executes less frequently. When long term scheduler selects a process from job pool, the state of process changes from new to ready state.

2. Short term scheduler: It is also known as CPU scheduler. This scheduler selects **processes that are ready for execution from the ready queue and allocates the CPU to the selected process.** Frequency of execution of short term scheduler is more than other schedulers. When short term scheduler selects a process, the state of process changes from ready to running state.

3. Medium term scheduler: When a process is in running state, due to some interrupt it is blocked. System swaps out blocked process and store it into a blocked and swapped out process queue. When space is available in the main memory, the operating system looks at the list of swapped out but ready processes. The medium term scheduler selects one process from that list and loads it into the ready queue. **The job of medium term scheduler is to select a process from swapped out process queue and to load it into the main memory.** This scheduler works in close communication with long term scheduler for loading process into the main memory

d) Explain two level directory structure with suitable diagram.

Ans. A **directory** is a container that is used to contain folders and files. It organizes files and folders in a hierarchical manner. In other words, directories are like folders that help organize files on a computer. Just like you use folders to keep your papers and documents in order, the operating system uses directories to keep track of files and where they are stored. Different structures of directories can be used to organize these files, making it easier to find and manage them.

Understanding these directory structures is important because it helps in efficiently organizing and accessing files on your computer. Following are the logical structures of a directory, each providing a solution to the problem faced in the previous type of directory structure.

Two-Level Directory

As we have seen, a single level directory often leads to confusion of files names among different users. The solution to this problem is to create a **separate directory for each user**.

In the two-level directory structure, each user has their own **user files directory (UFD)**. The UFDs have similar structures, but each lists only the files of a single user. System's **master file directory (MFD)** is searched whenever a new user id is created.

Advantages

- The main advantage is there can be more than two files with same name, and would be very helpful if there are multiple users.
- A security would be there which would prevent user to access other user's files.
- Searching of the files becomes very easy in this directory structure.

4. Attempt any THREE of the following: 12

a) Explain real time OS. Explain its types.

Ans. A real time system has well defined fixed time constraints. Processing should be done within the defined constraints -Hard and Soft real time system.

Real-time **operating systems (RTOS)** are used in environments where a large number of events, mostly external to the computer system, must be accepted and processed in a short time or within certain deadlines. such applications are industrial control, telephone switching equipment, flight control, and real-time simulations.

With an RTOS, the processing time is measured in tenths of seconds. This system is time-bound and has a fixed deadline. The processing in this type of system must occur within the specified constraints. Otherwise, This will lead to system failure.

Examples of real-time operating systems are airline traffic control systems, Command Control Systems, airline reservation systems, Heart pacemakers, Network Multimedia Systems, robots, etc.

A real-time operating system (RTOS) is a special kind of operating system designed to handle tasks that need to be completed quickly and on time. Unlike general-purpose operating systems (GPOS), which are good at multitasking and user interaction, RTOS focuses on doing things in real time.

Hard Real-Time Operating System

These operating systems guarantee that critical tasks are completed within a range of time. For example, a robot is hired to weld a car body. If the robot welds too early or too late, the car cannot be sold, so it is a hard real-time system that requires complete car welding by the robot hardly on time., scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

For Example,

Let's take an example of airbags provided by carmakers along with a handle in the driver's seat. When the driver applies brakes at a particular instance, the airbags grow and prevent the driver's head from hitting the handle. Had there been some delay even of milliseconds, then it would have resulted in an accident.

Soft Real-Time Operating System

This operating system provides some relaxation in the time limit. For example – Multimedia systems, digital audio systems, etc. Explicit, programmer-defined, and controlled processes are encountered in real-time systems. A separate process is changed by handling a single external event. The process is activated upon the occurrence of the related event signaled by an interrupt. Multitasking operation is accomplished by scheduling processes for execution independently of each other. Each process is assigned a certain level of priority that corresponds to the relative importance of the event that it services. The processor is allocated to the highest-priority processes. This type of schedule, called, priority-based preemptive scheduling is used by real-time systems.

For Example,

This type of system is used in Online Transaction systems and Livestock price quotation Systems.

Firm Real-time Operating System

RTOS of this type have to follow deadlines as well. In spite of its small impact, missing a deadline can have unintended consequences, including a reduction in the quality of the product. Example: Multimedia applications.

Example, this system is used in various forms of Multimedia applications.

Uses of RTOS

- Defense systems like [RADAR](#) .
- Air traffic control system.
- Networked multimedia systems.
- Medical devices like pacemakers.
- Stock trading applications.

Advantages

The advantages of real-time operating systems are as follows:

- **Maximum Consumption:** Maximum utilization of devices and systems. Thus more output from all the resources.
- **Task Shifting:** Time assigned for shifting tasks in these systems is very less. For example, in older systems, it takes about 10 microseconds. Shifting one task to another and in the latest systems, it takes 3 microseconds.
- **Focus On Application:** Focus on running applications and less importance to applications that are in the queue.
- **Real-Time Operating System In Embedded System:** Since the size of programs is small, RTOS can also be embedded systems like in transport and others.
- **Error Free:** These types of systems are error-free.
- **Memory Allocation:** Memory allocation is best managed in these types of systems.

Disadvantages

The disadvantages of real-time operating systems are as follows:

- **Limited Tasks:** Very few tasks run simultaneously, and their concentration is very less on few applications to avoid errors.
- **Use Heavy System Resources:** Sometimes the system resources are not so good and they are expensive as well.
- **Complex Algorithms :** The algorithms are very complex and difficult for the designer to write on.

b) Draw process state diagram and describe each state.

Ans. Different process states are as follows:

1. New
2. Ready
3. Running
4. Waiting
5. Terminated

New: When a process enters into the system, it is in new state. In this state a process is created. In new state the process is in job pool.

Ready: When the process is loaded into the main memory, it is ready for execution. In this state the process is waiting for processor allocation.

Running: When CPU is available, system selects one process from main memory and executes all the instructions from that process. So, when a process is in execution, it is in running state. In single user system, only one process can be in the running state. In multiuser system, there can be multiple processes which are in the running

state.

Waiting State: When a process is in execution, it may request for I/O resources. If the resource is not available, process goes into the waiting state. When the resource is available, the process goes back to ready state.

Terminated State:

When the process completes its execution, it goes into the terminated state. In this state the memory occupied by the process is released.

Operations on the Process

1. Creation

Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.

2. Scheduling

Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.

3. Execution

Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.

4. Deletion/killing

Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system.

c) Describe any four condition for deadlock.

Ans.By ensuring that at least one of below conditions cannot hold, we can prevent the occurrence of a deadlock.

1.Mutual Exclusion:

The mutual-exclusion condition must hold for non-sharable resources. Sharable resources do not require mutually exclusive access, thus cannot be involved in a deadlock.

2.Hold and Wait:

One way to avoid this Hold and Wait is when a process requests a resource; it does not hold any other resources.

â€¢One protocol that can be used requires each process to request and be allocated all its resources before it begins execution.

â€¢Another protocol that can be used is, to allow a process to request resources only when the process has none. A process may request some resources and use them. Before it requests any additional resources, it must release all the resources that are currently allocated to it.

3.No Preemption:

If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are preempted. That is these resources are implicitly released. The preempted resources are added to the list of resources for which the process is waiting. Process will be restarted only when all the resources i.e. its old resources, as well as the new ones that it is requesting will be available.

4.Circular Wait

Circular-wait condition never holds is to impose a total ordering of all resource types, and to require that each process requests resources in an increasing order of enumeration.

Let $R = \{R_1, R_2, \dots, R_n\}$ be the set of resource types. We assign to each resource type a unique integer number, which allows us to compare two resources and to determine whether one precedes another in our ordering. Formally, define a one-to-one function $F: R \rightarrow \mathbb{N}$, where $\mathbb{N}$ is the set of natural numbers.

d) Differentiate between paging and segmentation (any 4 points)

Ans.

Paging	Segmentation
In paging , the program is divided into fixed or mounted size pages.	In segmentation , the program is divided into variable size segments.
For the paging operating system is accountable.	For segmentation compiler is accountable.
Page size is determined by hardware.	Here, the segment size is given by the user.
It is faster in comparison to segmentation.	Segmentation is slow.
Paging could result in internal fragmentation.	Segmentation could result in external fragmentation.
In paging, the logical address is split into a page number and page offset.	Here, the logical address is split into segment number and segment offset.
Paging comprises a page table that encloses the base address of every page.	While segmentation also comprises the segment table which encloses segment number and segment offset.
The page table is employed to keep up the page data.	
In paging, the operating system must maintain a free frame list.	In segmentation, the operating system maintains a list of holes in the main memory.
Paging is invisible to the user.	Segmentation is visible to the user.
In paging, the processor needs the page number, and offset to calculate the absolute address.	In segmentation, the processor uses segment number, and offset to calculate the full address.

It is hard to allow sharing of procedures between processes.	Facilitates sharing of procedures between the processes.
In paging, a programmer cannot efficiently handle data structure.	It can efficiently handle data structures.
This protection is hard to apply.	Easy to apply for protection in segmentation.
The size of the page needs always be equal to the size of frames.	There is no constraint on the size of segments.
A page is referred to as a physical unit of information.	A segment is referred to as a logical unit of information.
Paging results in a less efficient system.	Segmentation results in a more efficient system.

e) Explain fixed and variable memory management.

Ans. The use of unequal size partitions provides a degree of flexibility to fixed partitioning. In dynamic partitioning, the partitions are of variable length and number. In noncontiguous memory allocation, a program is divided into blocks that the system may place in nonadjacent slots in main memory. This allocation method does not suffer from internal fragmentation, because a process partition is exactly the size of the process. Fig. 5.3.1 shows noncontiguous memory allocation method. Operating system maintains the table which contains the memory areas allocated to process and free memory. Memory management unit uses this information for allocating processes. Table contains information about memory starting address for process/program and their size. CPU sends the logical address of the process to the MMU and the MMU uses the memory allocation information stored in the table for calculating logical address. We call this address as effective memory address of the data/instruction.

Virtual memory is a method of using hard disk space to provide extra memory. It simulates additional main memory. In Windows operating system, the amount of virtual memory available equals the amount of free main memory plus the amount of disk space allocated to the swap file. Fig. 4.8.1 shows logical view of virtual memory concept. A swap file is an area of your hard disk that is set aside for virtual memory. Swap files can be either temporary or permanent. Virtual memory is stored in the secondary storage device. It helps to extend additional memory capacity and work with primary memory to load applications. The virtual memory will reduce the cost of expanding the capacity of physical memory.

The implementations of virtual memory will differ for operating systems to operating system. Each process address space is partitioned into parts that can be loaded into primary memory when they are needed and written back to secondary storage otherwise. Address space partitions have been used for the code, data and stack identified by the compiler and relocating hardware. The portion of the process that is actually in main memory at any time is defined to be the resident set of the process. The logical addressable space is referred to as virtual memory. The virtual address space is much larger than the physical primary memory in a computer system. The virtual memory works with the help of secondary storage device and its speed is low compared to the physical storage location.

5. Attempt any TWO of the following: 12

a) Explain the working of interprocess communication considering

i) Shared memory

ii) Message passing

Ans.1. Shared memory

In this, all processes who want to communicate with other processes can access a region of the memory residing in an address space of a process creating a shared memory segment. All the processes using the shared memory segment should attach to the address space of the shared memory. All the processes can exchange information by reading and/or writing data in shared memory segment.

The form of data and location are determined by these processes who want to communicate with each other.

These processes are not under the control of the operating system.

The processes are also responsible for ensuring that they are not writing to the same location simultaneously.

After establishing shared memory segment, all accesses to the shared memory segment are treated as routine memory access and without assistance of kernel.

2. Message Passing

In this model, communication takes place by exchanging messages between cooperating processes.

It allows processes to communicate and synchronize their action without sharing the same address space.

It is particularly useful in a distributed environment when communication process may reside on a different computer connected by a network.

Communication requires sending and receiving messages through the kernel.

The processes that want to communicate with each other must have a communication link between them. Between each pair of processes exactly one communication link.

b) With neat diagram explain multilevel queue scheduling.

Ans.Multilevel Queue Scheduling

Each algorithm supports a different process, but in a general system, some processes require scheduling using a priority algorithm. While some processes want to stay in the system (**interactive processes**), others are **background processes** whose execution can be delayed. The number of ready queue algorithms between queues and within queues may differ between systems. A round-robin method with various time quantum is typically utilized for such maintenance. Several types of scheduling algorithms are designed for circumstances where the processes can be readily separated into groups. There are two sorts of processes that require different scheduling algorithms because they have varying response times and resource requirements. **The foreground (interactive) and background processes** (batch process) are distinguished. Background processes take priority over foreground processes. The ready queue has been partitioned into seven different queues using the multilevel queue scheduling technique. These processes are assigned to one queue based on their priority, such as memory size, process priority, or type. The method for scheduling each queue is different. Some queues are utilized for the foreground process, while others are used for the background process. **The foreground queue may be scheduled using a round-robin method, and the background queue can be scheduled using an FCFS strategy.**

The main components of an MLQ architecture are

- **System Processes** – System processes refer to operating system tasks or processes that are essential for the proper functioning of the system. These processes handle critical operations such as memory management, process scheduling, I/O handling, and other system-level functions. System processes typically have the highest priority in the MLQ architecture and are executed before other types of processes.
- **Interactive Processes** – Interactive processes are user-oriented processes that require an immediate or near-real-time response. These processes are typically initiated by user interaction, such as keyboard input or mouse clicks. Examples of interactive processes include running a text editor, web browser, or a graphical application. Interactive processes are given priority in the MLQ architecture to ensure a smooth and responsive user experience.
- **Batch Processes** – Batch processes are non-interactive processes that are executed in the background without direct user interaction. These processes often involve executing a series of tasks or jobs that can be automated and executed without user intervention. Batch processes are typically resource-intensive and can include tasks such as data processing, large-scale computations, or scheduled backups. In the MLQ architecture, batch processes are allocated resources based on their priority, but they are generally given lower priority compared to interactive processes.

Different Levels of Queues

Processes are divided into many tiers of queues in multilevel queue scheduling according to their features, priority, and time limitations. The following various layers of queues can be utilized in multilevel queue scheduling:

- **Foreground Queue** – Interactive processes that demand quick system responses often use this queue. Usually, these procedures receive first attention
- **Background Queue** – This queue is used for non-interactive processes that take longer to complete. These tasks are prioritized less than foreground tasks.
- **Interactive queue** – The interactive queue is used for processes that demand a system response in a reasonable amount of time
- **Batch Queue** – This queue is used for batch processing, or processing many jobs at once. These tasks typically call for lengthy execution times and are submitted in advance.
- **System Queue** – This queue is used for system processes, such as device drivers and interrupt handlers, that need special permissions to complete their tasks. Usually, these procedures receive first attention.
- **Real-time Queue** – Real-time processes, which demand a prompt response from the system, are queued in this way. Usually, these procedures receive first attention.

Advantages

- Based on the characteristics of the processes, multilevel queue scheduling enables the operating system to distribute resources efficiently. As a result, the system's overall reaction time is decreased through effective resource utilization.
- The system responds quickly to operations with a higher priority, such as interactive processes, thanks to multilevel queue scheduling. This makes the system more responsive overall, which enhances the user experience.
- The operating system can prioritize operations according to their relevance via multilevel queue scheduling, ensuring that crucial tasks are carried out first.
- Multilevel queue scheduling can boost system throughput and performance concurrently scheduling tasks from various queues.

c) Write two uses of following operating system tools.

i) Security policy

ii) User Management

iii) Task scheduler

Ans.i) Security Policy:

A **security policy** in an operating system defines rules and guidelines for maintaining the security of the system and its resources. Here are four uses of security policies:

1. **Access Control:** Security policies specify which users or groups have access to specific system resources, such as files, directories, and applications, ensuring that only authorized users can interact with sensitive data.
2. **Authentication Enforcement:** Security policies define the requirements for user authentication, such as password complexity, expiration, multi-factor authentication (MFA), and account lockout mechanisms, enhancing system security.

3. **Network Security Configuration:** Security policies control network access by defining firewall rules, encryption protocols, and VPN usage. This protects the system from unauthorized external access and attacks.
4. **Audit and Monitoring:** Security policies govern how security events (such as login attempts, system access, and file modifications) are logged and monitored, helping system administrators detect potential security breaches and enforce compliance.

ii) User Management:

User management refers to the set of functions that administrators use to control user accounts and permissions within an operating system. Here are four uses of user management:

1. **Account Creation and Deletion:** User management allows administrators to create and remove user accounts, ensuring that only authorized personnel can access the system.
2. **Role-Based Access Control (RBAC):** Administrators can assign users to specific roles or groups (e.g., administrator, guest, or standard user), controlling the resources and actions they can access or perform based on their responsibilities.
3. **Password Management:** User management tools enable the setting and updating of passwords, enforcing password policies (e.g., strength and expiration), and allowing users to reset or recover passwords if forgotten.
4. **Session Monitoring and Termination:** User management allows administrators to monitor user sessions, including active login times, and terminate sessions if needed (e.g., for security reasons or to free up system resources).

iii) Task Scheduler:

A **task scheduler** in an operating system is used to schedule the execution of processes or programs at predefined times or intervals. Here are four uses of task schedulers:

1. **Automating Routine Maintenance:** Task schedulers can automatically run system maintenance tasks like disk defragmentation, backups, virus scans, and software updates at regular intervals, ensuring system health and security.
2. **Job Scheduling for Background Processes:** Task schedulers allow administrators to schedule long-running or batch jobs (e.g., database backups, report generation) to run during off-peak hours when system resources are more available.
3. **Trigger-Based Task Execution:** The task scheduler can initiate processes based on specific triggers (e.g., system startup, user logon, or reaching a certain condition), automating responses to system events without manual intervention.
4. **Periodic Task Execution:** For repetitive tasks (e.g., syncing data between servers or sending daily reports), the task scheduler can set up periodic execution at fixed intervals, saving time and ensuring regular task completion without manual input.

6. Attempt any TWO of the following: 12

a) List file allocation method and explain any one in details.

Ans. From the user's point of view, a file is an abstract data type. It can be created, opened, written, read, closed and deleted without any real concern for its

implementation. The implementation of a file is a problem for the operating system.

The main problem is how to allocate space to these files so that disk space is effectively utilized and files can be quickly accessed.

Three major methods of allocating disk space are in wide use:

- Contiguous
- Linked
- Indexed

1. Contiguous Allocation

In this scheme, each file occupies a contiguous set of blocks on the disk. For example, if a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be: $b, b+1, b+2, \dots, b+n-1$. This means that given the starting block address and the length of the file (in terms of blocks required), we can determine the blocks occupied by the file.

The directory entry for a file with contiguous allocation contains

- Address of starting block
- Length of the allocated portion.

The file 'mail' in the following figure starts from the block 19 with length = 6 blocks. Therefore, it occupies 19, 20, 21, 22, 23, 24 blocks.

Advantages:

- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the k th block of the file which starts at block b can easily be obtained as $(b+k)$.
- This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

Disadvantages:

- This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

2. Linked List Allocation

In this scheme, each file is a linked list of disk blocks which **need not be** contiguous. The disk blocks can be scattered anywhere on the disk.

The directory entry contains a pointer to the starting and the ending file block. Each block contains a pointer to the next block occupied by the file.

The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.

Advantages:

- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.

- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

Disadvantages:

- Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.
- It does not support random or direct access. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.

3. Indexed Allocation

In this scheme, a special block known as the **Index block** contains the pointers to all the blocks occupied by a file. Each file has its own index block. The ith entry in the index block contains the disk address of the ith file block. The directory entry contains the address of the index block as shown in the image:

Advantages:

- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

Disadvantages:

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.

b) For the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 Calculate the page faults applying.

i) Optimal

ii) LRU

iii) FiFo page

Replacement algorithms for a memory with three frames.

Ans.

c) Consider the four processes P1, P2, P3 and P4 with length of CPO Burst time. Find out Avg waiting time and Avg turn around time for the following Algorithms.

i) FCFS

ii) RR (Slice-4ms)

iii) SJF

Process Arrival time Burst time

P1 0 8

P2 1 4

P3 2 9

P4 3 5

Ans.